

Unidad 02 – Entorno de desarrollo y primeros pasos

Cátedra: Programación en Computación – UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

De qué se trata esta unidad

En la unidad 01 vimos **qué pasa** cuando corrés un programa. Hoy lo hacés vos.

Al terminar esta clase vas a poder:

-  Moverte por tu computadora **desde la terminal**, sin el explorador de archivos
-  Abrir el proyecto de la cátedra en VSCode y **hacerlo funcionar**
-  Escribir, guardar y **ejecutar** tu primer programa
-  Pedirle datos al usuario y **hacer algo** con ellos
-  Leer un mensaje de error y entender qué te está diciendo

 **La idea de fondo:** programar no es sólo escribir código. Es escribirlo *en algún lado*, guardarlo *con algún nombre*, y decirle a *algún programa* que lo ejecute. Esta clase es sobre ese "algún lado".

1. Las tres herramientas

Se confunden todo el tiempo, y son tres cosas distintas:

HERRAMIENTA	QUÉ ES	ANALOGÍA
Node.js	El intérprete . Lee tu archivo <code>.js</code> y lo ejecuta.	El motor
Visual Studio Code	El editor . Donde escribís el texto del programa.	El taller
La terminal	Donde das órdenes a la computadora escribiendo.	El tablero de mandos

Ninguna reemplaza a la otra:

- Podrías escribir el programa en el Bloc de notas y funcionaría igual: **VSCode no ejecuta nada**, sólo te ayuda a escribir.
- Podrías no abrir VSCode nunca: **Node no necesita el editor** para correr un archivo.
- Pero **sin Node no hay ejecución**. Es el único imprescindible.

 **Error #1 del curso:** creer que VSCode "corre" el programa. VSCode abre una terminal y le pide a Node que lo corra. El que trabaja es Node.

1.1 JavaScript no es Node.js

JavaScript es el lenguaje: las reglas de escritura. **Node.js** es un programa que entiende ese lenguaje y lo ejecuta en tu máquina.

Es la misma diferencia que hay entre el **idioma español** y una **persona que habla español**. El idioma no hace nada por sí solo; alguien tiene que hablarlo.

2. La terminal

2.1 Qué es

Una ventana donde escribís **comandos** y la computadora los ejecuta. Es la interfaz que existía antes de las ventanas y los íconos, y sigue existiendo porque para muchas tareas es **más rápida y más precisa**.

Cómo abrirla:

-  **Windows:** menú Inicio → escribí `cmd` → *Símbolo del sistema*
-  **macOS:** `Cmd + Espacio` → escribí `Terminal`
-  **Linux:** `Ctrl + Alt + T`

Vas a ver algo así, con el cursor esperando:

```
C:\Users\ivan>
```

Ese texto antes del cursor se llama **prompt** (no confundir con la función `prompt()` que vamos a usar más adelante) y te dice **en qué carpeta estás parado**.

2.2 La idea clave: siempre estás parado en una carpeta

Todo lo que escribas en la terminal se ejecuta **relativo a la carpeta donde estás**. Si le pedís que corra `app.js` y en esa carpeta no hay ningún `app.js`, no lo va a encontrar — aunque el archivo exista en otro lado de tu disco.



Esto explica el 80 % de los "no me anda" de las primeras clases.

2.3 Los cinco comandos que necesitas

QUÉ QUERÉS	WINDOWS (<code>CMD</code>)	MACOS / LINUX
Saber dónde estoy	<code>cd</code>	<code>pwd</code>
Ver qué hay acá	<code>dir</code>	<code>ls</code>
Entrar a una carpeta	<code>cd carpeta</code>	<code>cd carpeta</code>
Volver una carpeta atrás	<code>cd ..</code>	<code>cd ..</code>
Crear una carpeta	<code>mkdir carpeta</code>	<code>mkdir carpeta</code>

Ejemplo completo — crear una carpeta para la materia y entrar:

```
cd Documents
mkdir programacion
cd programacion
```

Después de eso, el prompt cambió: ahora estás **adentro** de `programacion`.

2.4 Rutas: absolutas y relativas

- **Relativa** — desde donde estás parado: `cd unidades`
- **Absoluta** — desde la raíz del disco, sin importar dónde estés:
 -  `cd C:\Users\ivan\Documents\programacion`
 -  `cd /Users/ivan/Documents/programacion`

`..` significa "**la carpeta de arriba**". Se pueden encadenar: `cd ../../` sube dos niveles.

2.5 ✨ El truco que más tiempo te ahorra

La tecla `Tab` **autocompleta**. Escribí las primeras letras de una carpeta y apretá `Tab`: la terminal completa el resto. Además de ser más rápido, **evita errores de tipeo**, que son la otra mitad de los "no me anda".

La flecha  trae el comando anterior. No lo vuelvas a escribir.

3. 🐛 Si la instalación falló

Verificá primero que Node quedó instalado:

```
node -v
```

```
v24.19.0
```

Y también `npm`, que viene junto con Node:

```
npm -v
```

```
11.6.2
```

Si eso funciona, saltá a la sección 4. Si no, buscá tu caso acá:

❌ `node no se reconoce como un comando interno o externo`

Node no quedó en el **PATH** — la lista de carpetas donde el sistema busca los programas.

1. **Cerrá la terminal y abrila de nuevo.** El PATH se lee al abrir; una terminal que ya estaba abierta cuando instalaste Node no lo ve. Esto resuelve la mayoría de los casos.
2. Si sigue, reinstalá Node y asegurate de dejar tildada la opción *"Add to PATH"*.
3. Como último recurso, reiniciá la máquina.

❌ **PowerShell:** `no se puede cargar el archivo npm.ps1`

Le pasa a casi todos en Windows. PowerShell bloquea la ejecución de scripts por defecto.

Solución rápida: usá el *Símbolo del sistema* (`cmd`) en lugar de PowerShell. En VSCode se elige desde el desplegable de la terminal.

Solución definitiva: está documentada con capturas en el [README del template](#)

(<https://github.com/italijancic/pc-2026/blob/main/template/README.md#-troubleshooting>).

❌ **Instalé la versión "Current" en vez de la LTS**

Andá a <https://nodejs.org/en/download> (<https://nodejs.org/en/download>), bajá la **LTS** e instalala encima. No hace falta desinstalar nada.

⚠ Usamos **LTS** (*Long Term Support*) toda la cursada para que todas las máquinas del curso se comporten igual.

4. Visual Studio Code

4.1 Abrir la carpeta correcta ⚠

Archivo → Abrir carpeta... y elegí la **carpeta raíz del proyecto**: la que contiene el `package.json`.

```
mi-proyecto/      ← 📁 ABRÍ ESTA
├─ package.json
├─ eslint.config.mjs
├─ src/
│  └─ app.js
│  └─ prompt.js
```

⚠ Si abris `src/` en lugar de `mi-proyecto/`, VSCode no ve el `package.json` y `npm run dev` va a fallar. Es el segundo error más frecuente del curso, y el mensaje que da no ayuda nada.

4.2 La terminal integrada

Ver → **Terminal**, o `Ctrl + ñ` (`Ctrl + `` en teclados en inglés).

Se abre una terminal **ya parada en la carpeta del proyecto**. Es la que vamos a usar siempre: te ahorra el `cd` y garantiza que estás en el lugar correcto.

4.3 Extensiones recomendadas

El repositorio de la cátedra ya trae la lista: cuando abris la carpeta, VSCode te ofrece instalarlas. Aceptá.

EXTENSIÓN	PARA QUÉ
ESLint	Te marca los errores de estilo y las fallas mientras escribís
Marp for VS Code	Ver las presentaciones de la cátedra

5. 📦 El proyecto de la cátedra

Todo el código del curso se escribe dentro de una copia del **template**, que está en `template/` (<https://github.com/italijancic/pc-2026/tree/main/template>) del repositorio.

5.1 Qué hay adentro

ARCHIVO / CARPETA	QUÉ ES
<code>package.json</code>	La ficha técnica del proyecto: nombre, dependencias y comandos disponibles
<code>src/app.js</code>	👉 Acá escribís vos . Es el archivo que se ejecuta
<code>src/prompt.js</code>	La función <code>prompt()</code> para leer datos del teclado. No lo toques
<code>eslint.config.mjs</code>	Las reglas de estilo con las que se corrige
<code>node_modules/</code>	Las dependencias descargadas. No se toca y no se versiona

5.2 `npm install`

La primera vez, y **sólo la primera vez** en cada copia del proyecto:

```
npm install
```

Lee las dependencias declaradas en `package.json`, las descarga de internet y las deja en `node_modules/`. Necesita conexión.

🔑 **Cómo saber si hace falta:** si **no ves** la carpeta `node_modules/`, corrélo. Si la ves, ya está.

5.3 `npm run dev`

Para correr tu programa:

```
npm run dev
```

Ese `dev` no es magia: está definido en el `package.json` y equivale a `node --watch src/app.js`.

El `--watch` significa que Node **queda mirando el archivo**: cada vez que guardás con `Ctrl + S`, lo vuelve a ejecutar solo. No hace falta volver a escribir el comando.

Para **cortarlo**: `Ctrl + C`.

5.4 Los dos comandos, y cuándo

```
npm install    # una sola vez, al empezar el proyecto
npm run dev    # cada vez que querés correr tu programa
```

6. 🙌 Tu primer programa

Abrí `src/app.js`, **borrá todo lo que tenga** y escribí:

```
console.log('Hola mundo')
```

Guardá (`Ctrl + S`) y corré `npm run dev`:

```
Hola mundo
```

Eso es un programa. Tiene una sola instrucción, y la computadora la ejecutó.

6.1 Qué es `console.log()`

Una **instrucción para mostrar algo en la terminal**. Se desarma así:

```
console.log('Hola mundo')
// ↑      ↑
// |      └─ el argumento: qué querés mostrar
// └─ la orden: "escribí esto en la consola"
```

Los **paréntesis** son obligatorios. Las **comillas** marcan dónde empieza y termina el texto.

6.2 Varios argumentos

Separados por comas. Se imprimen en la misma línea, con un espacio entre medio:

```
console.log('Tensión:', 220, 'V')
```

```
Tensión: 220 V
```

6.3 Texto con comillas simples, dobles o backticks

Las tres formas son válidas para escribir texto:

```
console.log('con comillas simples')
console.log("con comillas dobles")
console.log(`con backticks`)
```

En la cátedra usamos comillas simples, salvo cuando necesitamos un backtick (ya vas a ver por qué). Lo importante: **la que abre tiene que ser la misma que cierra**.

6.4 ✨ Los backticks: meter valores adentro del texto

Con backticks (```) podés insertar valores dentro del texto usando `${...}`:

```
const tension = 220
console.log(`La tensión de línea es ${tension} V`)
```

```
La tensión de línea es 220 V
```

Esto se llama **template literal** (*plantilla de texto*), y lo vas a usar durante toda la cursada: es mucho más legible que ir cortando el texto con comas.

Compará:

```
// 😞 Con comas
console.log('La tensión de línea es', tension, 'V')

// 😊 Con template literal
console.log(`La tensión de línea es ${tension} V`)
```

⚠ El backtick **no es** una comilla simple. En un teclado español está a la izquierda del **1**, o junto a la **P**. Si usás **'** en lugar de **`**, el `${tension}` se imprime tal cual, literal.

7. 💬 Comentarios

Texto que **la computadora ignora**. Sirven para explicarle el código a un humano — casi siempre, a vos mismo dentro de tres semanas.

```
// Un comentario de una línea

/*
  Un comentario
  de varias líneas
*/

console.log('Esto sí se ejecuta') // también se puede al final de una línea
```

También sirven para **desactivar** temporalmente una línea sin borrarla:

```
// console.log('Esta línea no se ejecuta')
```

8. 📦 Guardar un dato: variables

Para que un programa haga algo interesante necesita **recordar valores**. Una **variable** es un nombre que le ponés a un dato:

```
const tension = 220
const material = 'cobre'

console.log(tension)
console.log(material)
```

```
220
cobre
```

Se lee así: "creá un espacio llamado `tension` y guardá adentro el valor `220`".

A partir de ahí, escribir `tension` en cualquier parte del programa **es lo mismo que escribir `220`**:


```
const tension = 220
const corriente = 5

console.log(`Potencia: ${tension * corriente} W`)
```

```
Potencia: 1100 W
```

✂ Por ahora usamos `const` y alcanza. **En la unidad 03** vemos las diferencias entre `let`, `const` y `var`, las reglas para nombrarlas y los operadores en detalle.

9. 📄 Pedirle datos al usuario

Hasta acá los valores estaban **escritos en el código**. Si querés calcular la potencia de otro motor, tenés que editar el programa. Eso no es un programa útil: es una calculadora de un solo uso.

Para leer datos del teclado usamos la función `prompt()`, que viene en `src/prompt.js`. Lo único que hay que hacer es **importarla al principio del archivo**:

```
import { prompt } from './prompt.js'

const nombre = prompt('¿Cómo te llamás? ')
console.log(`Hola ${nombre}, bienvenido al curso`)
```

```
¿Cómo te llamás? Ana
Hola Ana, bienvenido al curso
```

El programa **se detiene** en el `prompt()` y espera a que escribas algo y presiones `Enter`.

⚠ La línea `import` va **siempre al principio del archivo**, antes de todo lo demás.

9.1 ⚠ La trampa: todo entra como texto

`prompt()` devuelve **siempre texto**, aunque escribas un número. Y para el texto, el operador `+` no suma: **pega**.

```
import { prompt } from './prompt.js'

const a = prompt('Primer número: ')
const b = prompt('Segundo número: ')

console.log(a + b)
```

```
Primer número: 20
Segundo número: 5
205
```

205 . No sumó $20 + 5$: pegó el texto `'20'` con el texto `'5'` .

9.2 La solución: `parseInt()` y `parseFloat()`

Convierten texto a número:

FUNCIÓN	CONVIERTE A	EJEMPLO
<code>parseInt()</code>	Número entero	<code>parseInt('25')</code> → 25
<code>parseFloat()</code>	Número con decimales	<code>parseFloat('1.75')</code> → 1.75

```
import { prompt } from './prompt.js'

const a = parseInt(prompt('Primer número: '))
const b = parseInt(prompt('Segundo número: '))

console.log(a + b)
```

```
Primer número: 20
Segundo número: 5
25
```

Ahora sí. 🎉

🔑 **La regla:** si el dato es un número y lo vas a usar para calcular, envólvé el `prompt()` en `parseInt()` o `parseFloat()` . **Siempre.**

Usá `parseFloat()` ante la duda: una temperatura, una tensión o una longitud rara vez son enteras. `parseInt('21.5')` te devuelve `21` y perdés el decimal **sin ningún aviso**.

10. ¹²/₃₄ Los decimales que no pediste

Pasar una temperatura de Celsius a Kelvin es sumarle 273,15. Probemos:

```
const celsius = 21.7
const kelvin = celsius + 273.15

console.log(`${kelvin} K`)
```

```
294.84999999999997 K
```

Debería dar **294,85**. Y sin embargo aparecen catorce decimales de basura.

No es un error tuyo. Es la precisión finita que vimos en la unidad 01, con el famoso `0.1 + 0.2`: la computadora guarda los números con una cantidad limitada de bits, y hay decimales que en binario **no tienen representación exacta**. El error es minúsculo, del orden de 10^{-14} , pero se ve al imprimir.

La solución: `.toFixed()`

Le pedís el número de decimales que querés mostrar:

```
const celsius = 21.7
const kelvin = celsius + 273.15

console.log(`${kelvin.toFixed(2)} K`)
```

```
294.85 K
```

`.toFixed(2)` **redondea a dos decimales**. Se escribe pegado al número, con un punto:

ESCRIBÍS

SE MUESTRA

```
(294.84999999999997).toFixed(2)
```

```
294.85
```

```
(1596).toFixed(2)
```

```
1596.00
```

```
(4.2).toFixed(1)
```

```
4.2
```

🔑 **La regla práctica:** hacé todas las cuentas con los números completos y usá `.toFixed()` **sólo al mostrar**. Si redondeás en el medio, los errores se te acumulan.

11. 🐛 Leer un mensaje de error

Los errores no son un castigo: son **la información más útil que te da la computadora**. Casi siempre te dicen el archivo, la línea y qué pasó.

`SyntaxError` — **está mal escrito**

```
SyntaxError: missing ) after argument list
```

Node **ni siquiera empezó a ejecutar**: no entiende lo que escribiste. Casi siempre es un paréntesis, una llave o una comilla sin cerrar.

ReferenceError – nombraste algo que no existe

```
ReferenceError: tencion is not defined
```

Escribiste `tencion` y la variable se llama `tension`. 🖱️ **Revisá cómo lo escribiste.**

Cannot find module

```
Error: Cannot find module '/Users/ivan/src/app.js'
```

Node buscó el archivo y no está ahí. 🖱️ **Estás parado en la carpeta equivocada**, o el archivo tiene otro nombre.

Cannot find package 'readline-sync'

Falta `npm install`.

🔑 **Cómo leer un error:** empezá por la **primera línea** (dice qué pasó) y buscá el **número de línea** de tu archivo. El resto del texto casi siempre es ruido interno de Node.

12. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
Abrir <code>src/</code> en vez de la raíz	<code>npm run dev</code> no encuentra el script	Abrí la carpeta que tiene el <code>package.json</code>
No correr <code>npm install</code>	<code>Cannot find package 'readline-sync'</code>	Corré <code>npm install</code>
Terminal en otra carpeta	<code>Cannot find module</code>	Usá la terminal integrada de VSCode
Olvidar <code>parseInt</code> / <code>parseFloat</code>	<code>20 + 5</code> da <code>205</code>	Convertí el texto a número
Confundir <code>`</code> con <code>'</code>	Se imprime <code>\${tension}</code> tal cual	Usá backtick para los <code>\${}</code>
Decimales de más	<code>294.84999999999997</code>	Mostralo con <code>.toFixed(2)</code>
No guardar el archivo	Corrés y no cambia nada	<code>Ctrl + S</code> antes de ejecutar
Cerrar la terminal para "parar"	Queda corriendo	<code>Ctrl + C</code>
<code>node -v</code> no responde	<code>no se reconoce como comando</code>	Cerrá y reabrí la terminal

13. 📄 Resumen

CONCEPTO	PARA QUÉ
<code>node -v</code> · <code>npm -v</code>	Verificar que el entorno está instalado
<code>cd</code> · <code>dir</code> / <code>ls</code> · <code>mkdir</code>	Moverte por las carpetas desde la terminal
<code>npm install</code>	Descargar las dependencias. Una vez por proyecto
<code>npm run dev</code>	Ejecutar tu programa. Cada vez
<code>Ctrl + C</code>	Cortar un programa que está corriendo
<code>console.log(...)</code>	Mostrar algo en la terminal
<code>`texto \${valor}`</code>	Insertar un valor dentro de un texto
<code>//</code> · <code>/* */</code>	Comentarios: los ignora la computadora
<code>const nombre = valor</code>	Guardar un dato con un nombre
<code>prompt('...')</code>	Leer del teclado. Devuelve texto
<code>parseInt</code> · <code>parseFloat</code>	Convertir ese texto a número
<code>.toFixed(2)</code>	Mostrar con la cantidad de decimales que querés

14. 🚀 Para la próxima clase

En la **unidad 03** entramos en serio en el lenguaje: `let`, `const` y `var` y en qué se diferencian, las convenciones para nombrar variables, los tipos de datos y todos los operadores.

Para llegar preparado:

1. ✅ Dejá el template funcionando: `npm run dev` tiene que correr sin errores
2. ✅ Hacé el TP de esta unidad — es **la** práctica que necesitás para que la 03 no se te haga cuesta arriba
3. ✅ Si algo no te anduvo, **traelo anotado**

📎 Material de la unidad

- [Presentación de clase](https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/presentacion.pdf) (<https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/presentacion.pdf>)

- [Trabajo Práctico](https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/tp.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/02-entorno-y-primeros-pasos/tp.pdf)
- [Template del curso](https://github.com/italijancic/pc-2026/tree/main/template) (https://github.com/italijancic/pc-2026/tree/main/template)